InEar Teensy Compact Protocol

Technical Documentation

Version 1.0 — February 2026

Abstract

This document describes the Compact Protocol, a binary streaming format used by the InEar EEG system to transmit 8-channel electroencephalography data and auxiliary sensor readings from a Teensy 4.1 microcontroller to a host application over a 2 Mbaud serial link. It is written for engineers implementing parsers, firmware modifications, or analysis pipelines, and focuses on how the protocol works, why it is designed the way it is, and what an implementer needs to know.

Contents

1 Introduction

The InEar EEG device streams continuous neural data from an ADS1299 analog front-end alongside readings from an accelerometer, a photoplethysmography (PPG) sensor, a temperature sensor, and a battery monitor. All of these must travel over a single serial connection at 1000 samples per second with minimal latency.

The Compact Protocol was designed to meet three constraints simultaneously:

1. **Low overhead.** Every unnecessary byte costs bandwidth on a 2 Mbaud link. The protocol uses 8 bytes of framing (header + checksum) per packet, yielding 25% overhead on the most common 32-byte EEG-only packet.
2. **Deterministic parsing.** The host must be able to locate packet boundaries reliably, even after partial data loss. A two-byte sync word combined with a known packet length (derived from a single type field) makes this possible without footer bytes or escape sequences.
3. **TTL event support.** Experimental triggers must be timestamped at EEG-sample resolution. Rather than dedicating a separate byte, the protocol embeds two TTL bits directly into the type field.

The result is a variable-length packet between 32 and 38 bytes that carries one EEG sample, optional auxiliary data from at most one additional sensor, and two TTL trigger bits—all protected by a single-byte XOR checksum.

2 Architecture

2.1 Data Flow

The data path from sensor to analysis tool passes through three stages:

1. **Firmware (Teensy 4.1).** The ADS1299 signals a data-ready interrupt at 1000 Hz. The interrupt handler reads 24 bytes of EEG data over SPI, polls auxiliary sensors according to a scheduling table, assembles a binary packet, and writes it to the USB-serial transmit buffer.

2. **Host application (Open Ephys / Qt).** A parser thread reads the serial port, synchronizes on the `0xA5 0x5A` header, extracts fields, verifies the checksum, and pushes samples into a ring buffer.
3. **Lab Streaming Layer (LSL).** An LSL outlet publishes the parsed EEG data as 32-bit floating-point microvolts and TTL events as string markers, making them available to LabRecorder, MNE-Python, or any other LSL consumer.

2.2 Byte Order and Encoding

All multi-byte fields are transmitted in **big-endian** (network) order, which matches the native output format of the ADS1299. This avoids byte-swapping on the firmware side. The host parser must convert to the platform's native order when reading timestamps and sensor values.

3 Packet Format

Every packet follows a single layout. The only variable part is an optional auxiliary block whose presence and size are determined by a 3-bit type code.

SYNC `A5 5A` 2 B	**TIMESTAMP** uint32 BE 4 B	**TYPE +TTL** 1 B	**EEG** — 8 ch × 3 B = 24 B	**[AUX]** 0–6 B	**CHK** XOR 1 B

Figure 1: General packet layout. The AUX block is absent in EEG-only packets.

3.1 Sync Word (Bytes 0–1)

Every packet begins with the two-byte sequence `0xA5 0x5A`. This pattern was chosen because it alternates bit polarities (10100101 01011010), making it unlikely to appear accidentally in EEG or sensor data. If the parser loses synchronization—for instance after a corrupted packet—it scans the incoming byte stream for this marker to re-align.

Because there is no footer or end-of-packet delimiter, the sync word is the *only* mechanism for finding packet boundaries. This is sufficient because the parser knows the exact packet length once it reads the type code at byte 6.

3.2 Timestamp (Bytes 2–5)

A 32-bit unsigned integer recording the number of microseconds elapsed since the Teensy booted. At 1000 Hz, consecutive packets are nominally 1000 μs apart. The timestamp serves two purposes:

- **Ordering.** Packets arrive in order over a serial link, but the timestamp makes the ordering explicit and verifiable.
- **Loss detection.** If the difference between two consecutive timestamps exceeds approximately 1000 μs (plus jitter tolerance), the host knows one or more packets were lost.

There is no separate sequence counter. The microsecond timestamp provides strictly monotonic ordering and finer-grained gap detection than a simple integer counter would.

Note: The 32-bit microsecond counter wraps after roughly 71.6 minutes ($2^{32}/10^6/60$). Parsers must handle this rollover gracefully.

3.3 TYPE+TTL Byte (Byte 6)

A single byte encodes three pieces of information:

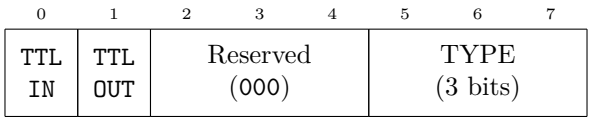

Figure 2: Bit-field layout of the TYPE+TTL byte. Bit 7 is the MSB.

- **Bits 7–6: TTL lines.** Bit 7 reflects the state of the external trigger input; bit 6 reflects the trigger output. These are sampled at the same instant as the EEG data, giving sample-accurate event timing without a separate event channel.
- **Bits 5–3: Reserved.** Must be `000` under normal operation. When all three are set to `1`, the packet signals a firmware error (see section 5).
- **Bits 2–0: Packet type.** Determines which auxiliary sensor, if any, is included in the packet.

Merging TTL state into the type byte saves one byte per packet compared to a dedicated TTL field. Over one second of streaming at 1000 Hz, that is 1000 bytes—a meaningful saving on a serial link.

3.3.1 Type Codes

The 3-bit type field supports eight codes. Five are currently assigned:

Code	Name	Auxiliary Payload	Packet Size
0	EEG only	—	32 B
1	EEG + Accel	3× int16 BE (6 B)	38 B
2	EEG + PPG	uint24 BE + uint8 (4 B)	36 B
3	EEG + Temp	int16 BE (2 B)	34 B
4	EEG + Battery	uint8 (1 B)	33 B

Codes 5–7 are reserved for future expansion. A parser encountering an unknown type code should discard the packet and attempt to resynchronize.

3.3.2 Extracting Fields

Given the raw byte value, the individual fields are obtained with bit masking:

$$\begin{aligned} \texttt{ttl\_in} &= (\texttt{byte} \gg 7)\ \&\ 1 \\ \texttt{ttl\_out} &= (\texttt{byte} \gg 6)\ \&\ 1 \\ \texttt{reserved} &= (\texttt{byte} \gg 3)\ \&\ 7 \\ \texttt{type} &= \texttt{byte}\ \&\ 7 \end{aligned}$$

3.4 EEG Data (Bytes 7–30)

The ADS1299 produces eight channels of 24-bit signed data per conversion cycle. These 24 bytes are copied directly from the SPI read buffer into the packet with no transformation. Each channel occupies three bytes in big-endian two's-complement format.

To convert a raw 24-bit value to a voltage in microvolts:

$$V_{\mu\text{V}} = \frac{\text{raw}_{24} \times 4.5}{2^{23} \times G} \times 10^6$$

where G is the programmable gain (typically 24 for EEG). With gain 24, the least-significant bit corresponds to approximately $0.02235\,\mu$V.

Note: Sign extension is required when unpacking into a 32-bit integer. If bit 23 of the raw value is set, the upper byte must be filled with `0xFF`.

3.5 Auxiliary Data (Bytes 31–$n-2$)

If the type code is nonzero, an auxiliary block immediately follows the EEG data. The block's format depends on the type:

3.5.1 Accelerometer (Type 1, 6 bytes)

Three 16-bit signed integers representing the X, Y, and Z axes of an ADXL367 accelerometer. At the default $\pm 2\,g$ range and 14-bit resolution, each LSB represents 0.25 mg. The accelerometer is sampled at 200 Hz, so this block appears in every 5th packet.

3.5.2 PPG + SpO_2 (Type 2, 4 bytes)

A 24-bit unsigned PPG intensity value from the MAXM86161 optical sensor, followed by a single byte representing a firmware-computed SpO_2 estimate (0–100, percent). PPG data arrives at 100 Hz (every 10th packet).

3.5.3 Temperature (Type 3, 2 bytes)

A 16-bit signed value from the TMP117 digital temperature sensor. Each LSB equals 0.0078125 °C (1/128), giving a resolution of ∼0.008 °C. Temperature is sampled at 1 Hz.

3.5.4 Battery (Type 4, 1 byte)

A single unsigned byte representing the battery voltage level, read through a voltage divider. Sampled at 1 Hz.

3.6 Checksum (Final Byte)

The last byte of every packet is the XOR of all preceding bytes (offsets 0 through $n-2$):

$$\text{checksum} = \bigoplus_{i=0}^{n-2} \text{packet}[i]$$

XOR was chosen over CRC for its simplicity and speed on the Teensy's ARM Cortex-M7. It detects any single-bit error and most multi-bit errors, which is sufficient for a short packet on a reliable USB-serial link.

To verify: XOR all bytes including the checksum. The result should be zero.

3.7 Byte Offset Summary

For quick reference, the absolute byte offsets for each packet type:

Field	Type 0 EEG	Type 1 +Accel	Type 2 +PPG	Type 3 +Temp	Type 4 +Bat
Sync (2 B)	0–1	0–1	0–1	0–1	0–1
Timestamp (4 B)	2–5	2–5	2–5	2–5	2–5
Type+TTL (1 B)	6	6	6	6	6
EEG (24 B)	7–30	7–30	7–30	7–30	7–30
Aux data	—	31–36	31–34	31–32	31
Checksum (1 B)	31	37	35	33	32
Total	32 B	38 B	36 B	34 B	33 B

4 Sensor Scheduling

The EEG front-end generates a data-ready interrupt every millisecond. Each interrupt produces exactly one packet. Auxiliary sensors run at lower rates, so most packets carry EEG data alone (type 0). The scheduling rules determine when each auxiliary sensor gets to attach its data to a packet.

4.1 Rates and Rationale

Sensor	Rate	Period	Why
EEG (+ TTL)	1000 Hz	1 ms	Neural signals require high temporal resolution
Accelerometer	200 Hz	5 ms	Head movement artifacts; motion tracking
PPG + SpO_2	100 Hz	10 ms	Cardiac cycle bandwidth (∼25 Hz)
Temperature	1 Hz	1000 ms	Body temperature changes on the order of minutes
Battery	1 Hz	1000 ms	Voltage changes slowly over hours

At these rates, approximately 80% of all packets are EEG-only (type 0), 20% carry an accelerometer reading, 10% carry PPG, and temperature and battery each appear once per second. Because accelerometer and PPG periods share a common multiple (every 10th packet), they collide regularly and require a resolution mechanism.

4.2 The One-Auxiliary-Per-Packet Rule

A fundamental constraint of the Compact Protocol is that each packet carries data from **at most one** auxiliary sensor in addition to EEG. This keeps the packet format simple—the 3-bit type code unambiguously identifies the payload layout—and avoids large combined packets that would increase worst-case latency.

The trade-off is that when two sensors are due simultaneously, one of them must be delayed.

4.3 Collision Resolution by Deferral

When multiple auxiliary sensors are due on the same EEG sample, the firmware applies a fixed priority:

$$\text{Accelerometer} > \text{PPG} > \text{Temperature} > \text{Battery}$$

The highest-priority sensor claims that packet. Every other sensor that was due is **deferred by exactly 1 ms**—that is, it is sent in the next packet, which corresponds to the next EEG sample. The deferred packet is a completely normal packet of the appropriate type. Crucially, it carries the **fresh EEG sample from the deferred time slot**, not a copy of the previous sample. No EEG data is ever duplicated or skipped.

Example. At $t = 10\,\text{ms}$, both the accelerometer (due every 5 ms) and PPG (due every 10 ms) want to send. The accelerometer wins. The packet at $t = 10\,\text{ms}$ is type 1 (EEG + Accel) with EEG data from $t = 10$. The PPG is deferred to $t = 11\,\text{ms}$, producing a type 2 (EEG + PPG) packet with EEG data from $t = 11$.

This approach ensures that the EEG stream remains perfectly regular—one fresh sample per millisecond, every millisecond—regardless of how auxiliary sensors are scheduled.

4.4 Staggering the 1 Hz Sensors

Temperature and Battery both run at 1 Hz. If both were due at the same second boundary they would always collide. To prevent this, they are staggered with a fixed offset so that only one 1 Hz sensor fires at any given second.

Note: The exact staggering offset between Temperature and Battery is not yet finalized in the firmware. The only firm constraint is that they must not be due on the same sample. If either 1 Hz sensor coincides with an accelerometer or PPG packet, the standard deferral rule applies.

4.5 What This Means for Parsers

A parser should not assume that auxiliary data arrives at perfectly regular intervals. Due to deferral, a PPG reading expected at $t = 10\,\text{ms}$ may arrive at $t = 11\,\text{ms}$. The correct approach is to use each packet's timestamp to determine when the data was actually sampled, rather than inferring timing from packet sequence position.

5 Error Handling

The protocol provides two error-detection mechanisms and one error-signaling mechanism.

5.1 Checksum Verification

The XOR checksum catches transmission errors. On checksum failure, the host should:

1. Discard the corrupted packet.
2. Increment an error counter for diagnostics.
3. Continue reading—the next sync word marks the start of the following packet.

A single checksum failure is not cause for disconnection. However, a burst of consecutive failures may indicate a hardware problem (loose connection, baud rate mismatch).

5.2 Sync Loss Recovery

If the parser encounters bytes that do not form a valid packet—for example, an unexpected byte where a sync word was expected—it must scan forward through the stream, byte by byte, looking for the `0xA5 0x5A` pattern. Once found, the parser reads the type byte at offset 6, determines the expected packet length, reads that many bytes, and verifies the checksum. If the checksum passes, synchronization has been restored.

Note: The sync pattern can theoretically appear inside EEG data. In practice this is extremely rare. The checksum provides a secondary confirmation that the parser is truly aligned. If the sync word is found but the checksum fails, the parser should skip that candidate and continue scanning.

5.3 Firmware Error Signal

When the firmware detects an unrecoverable error (e.g., SPI communication failure with the ADS1299), it sends a special error packet. This packet has the normal header structure, but the reserved bits (5–3) in the TYPE+TTL byte are all set to `1`, and the EEG data region is zeroed out:

`0xA5 0x5A` + Timestamp (4 B) + *error-flagged TYPE+TTL* + 24×`0x00` + Checksum

Any TYPE+TTL byte where (`byte` $\gg 3$) & $7 = 7$ is an error signal. Upon receiving it, the host must:

1. Stop data acquisition.
2. Transition back to the handshake / connection state.
3. Report the error to the user.

6 Bandwidth and Timing

6.1 Serial Link Budget

The serial link runs at 2,000,000 baud. With 8N1 framing (10 bits per byte), the effective byte rate is 200,000 B/s. The average data rate depends on the mix of packet types:

$$\begin{aligned}\text{Average B/s} &\approx 698 \times 32 + 200 \times 38 + 100 \times 36 + 1 \times 34 + 1 \times 33 \\ &= 22{,}336 + 7{,}600 + 3{,}600 + 34 + 33 \\ &\approx 33{,}603\,\text{B/s}\end{aligned}$$

This represents **16.8%** of the available bandwidth, leaving 83.2% headroom. The generous margin accommodates occasional retransmissions, USB scheduling jitter, and potential future expansion (more channels or higher auxiliary rates).

6.2 Worst-Case Burst

The largest packet is 38 bytes (EEG + Accelerometer). At 200,000 B/s, transmitting one such packet takes 0.19 ms, well within the 1 ms inter-sample interval. Even two back-to-back large packets (e.g., accelerometer at t followed by deferred PPG at $t + 1$) total 74 bytes, requiring 0.37 ms—still comfortably within budget.

6.3 Overhead Efficiency

Every packet carries 8 bytes of fixed overhead (2 B sync + 4 B timestamp + 1 B type + 1 B checksum). As a fraction of the total packet:

EEG Channels	EEG Data	EEG-Only Packet	Overhead
5	15 B	23 B	34.8%
8	24 B	32 B	25.0%
12	36 B	44 B	18.2%
16	48 B	56 B	14.3%

The 8-byte overhead becomes proportionally cheaper as the channel count increases. At 8 channels the protocol is already reasonably efficient at 25%.

7 Integration with Lab Streaming Layer

The host application publishes parsed data over Lab Streaming Layer (LSL), which is the standard transport for real-time neural data in research environments. Two streams are created: an EEG data stream and a marker event stream.

7.1 EEG Stream

The EEG stream publishes 32-bit floating-point values in microvolts at a nominal rate of 1000 Hz. The conversion from raw 24-bit ADC counts to microvolts happens in the host parser before data enters LSL.

Key stream properties:

Property	Value	Rationale
`name`	`"InEarEEG"`	Human-readable; unique per device
`type`	`"EEG"`	Recognized by LabRecorder, MNE-Python, EEGLAB
`channel_count`	8	Matches hardware configuration
`nominal_srate`	1000.0	Must match actual firmware rate
`channel_format`	`cf_float32`	Data pushed as microvolts
`source_id`	`"inear_001"`	Stable across sessions; unique per device

Each channel's XML metadata should include a `label` (ideally 10–20 system names like Fp1, Cz), a `unit` of `"microvolts"` (not `"uV"`—the spelled-out form is the LSL/XDF convention), and a `type` of `"EEG"`.

The `acquisition` XML block should include `manufacturer`, `model`, `serial_number`, and a `scale_factor` of approximately 0.02235 (μV per LSB at gain 24).

7.2 Marker Stream

TTL events are published on a separate irregular-rate string stream. This stream uses `type = "Markers"`, `nominal_srate = 0` (irregular), and `channel_format = cf_string`.

Recommended marker strings:

Event	String
TTL input rising edge	`"TTL_IN_1"`
TTL input falling edge	`"TTL_IN_0"`
TTL output rising edge	`"TTL_OUT_1"`
TTL output falling edge	`"TTL_OUT_0"`

7.3 Outlet Configuration

The LSL outlet should use a chunk size of $\lfloor \text{srate}/20 \rfloor = 50$ samples, which corresponds to approximately 50 ms of data per push. This balances latency (small enough for near-real-time feedback) against efficiency (large enough to avoid per-sample overhead). A `max_buffered` value of 360 seconds provides a 6-minute buffer to absorb brief network or processing stalls.

8 Design Rationale

This section explains the reasoning behind several non-obvious design choices.

8.1 Why No Footer?

Many serial protocols use a footer (e.g., `0xC0 0xC0`) to mark the end of a packet. The Compact Protocol omits this because:

- The packet length is fully determined by the 3-bit type code. Once the parser reads the type, it knows exactly how many bytes to expect.
- Eliminating 2 footer bytes per packet saves 2000 bytes per second at 1000 Hz—a non-trivial reduction.
- Footer bytes require escaping if the same byte pattern appears in the payload, adding complexity. The Compact Protocol avoids this entirely.

8.2 Why No Sequence Counter?

A sequence counter (typically 1–4 bytes) is common in streaming protocols. Here, the 4-byte microsecond timestamp already provides:

- Strict monotonic ordering (the same guarantee a counter provides).
- Loss detection with sub-millisecond precision (a counter can only detect that *something* was lost, not *when*).
- Absolute timing information that a counter cannot provide.

Since the timestamp already occupies 4 bytes and satisfies all use cases that a counter would, adding a separate counter would be pure redundancy.

8.3 Why XOR Instead of CRC?

XOR is the simplest possible checksum. It detects any single-bit error but can miss certain multi-bit error patterns (specifically, any even number of errors in the same bit position across bytes). A CRC-8 or CRC-16 would provide stronger detection.

XOR was chosen because the USB-serial link already has its own CRC at the USB layer, making transport errors extremely rare. The protocol-level checksum primarily guards against firmware bugs (e.g., a buffer overrun corrupting a packet) rather than transmission errors. For this purpose, XOR is sufficient and costs only a few CPU cycles.

8.4 Why Variable-Length Packets?

An alternative design would use a fixed packet size (e.g., always 38 bytes) with unused auxiliary fields zeroed out. Variable-length packets were preferred because:

- 80% of all packets carry no auxiliary data. Padding each of these with 6 unused bytes would waste $0.8 \times 6 \times 1000 = 4800$ bytes per second.
- Variable-length packets make the type code semantically meaningful: if the type says "EEG only," there is genuinely nothing else in the packet. This simplifies debugging.

The cost is a slightly more complex parser that must determine packet length from the type code before reading the remainder.

9 Implementation Notes

9.1 Parser State Machine

A robust parser typically uses three states:

1. **SEEKING_SYNC.** Scan bytes one at a time, looking for `0xA5` followed by `0x5A`.
2. **READING_HEADER.** Read the next 5 bytes (timestamp + type). Determine packet length from the type code. If the type code is invalid (5–7), return to SEEKING_SYNC.

3. **READING_BODY.** Read the remaining bytes (EEG + optional aux + checksum). Verify the checksum. If valid, emit the packet; if invalid, return to SEEKING_SYNC and scan from the byte after the sync word that failed.

9.2 Sign Extension for 24-Bit EEG

The ADS1299's 24-bit output is signed (two's complement). When unpacking into a 32-bit integer:

1. Assemble the three bytes: raw $= (b_0 \ll 16) \mid (b_1 \ll 8) \mid b_2$.
2. Check the sign bit: if raw & `0x800000` $\neq 0$, set the upper byte: raw $\mid =$ `0xFF000000`.

Failing to sign-extend will cause negative EEG values to be interpreted as large positive numbers, producing obvious artifacts in the signal.

9.3 Timestamp Rollover

The 32-bit microsecond timestamp rolls over at $2^{32} - 1 = 4{,}294{,}967{,}295\,\mu$s (≈71.6 minutes). To compute the interval between two timestamps t_1 and t_2 (where t_2 may have wrapped):

$$\Delta t = (t_2 - t_1) \mod 2^{32}$$

Standard unsigned subtraction on 32-bit integers handles this automatically in C and most languages.

9.4 Handling Unknown Type Codes

If the parser encounters a type code of 5, 6, or 7, it cannot determine the packet length. The safest response is to discard the current parse attempt and return to SEEKING_SYNC. This situation should be logged, as it may indicate firmware/parser version mismatch.